Threat Modelling in Practice

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

Threat Modelling: Lecture Overview

Why Model at All: what an inventory does not tell you, and why the order of modelling and attacking matters

Data-Flow Diagrams: four elements, trust boundaries, and the course VM drawn as a diagram

STRIDE: six questions asked of every element, the grid, and three worked rows

Attack Trees and Priorities: decomposing one goal, reading off the cheapest path, and ranking what you found

Into the Lab: from model to policy, the surface as a number, and lab report 1

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

Why Model at All

- You left Lab 1 with an inventory: what listens, what is privileged, what is scheduled, what is exposed at rest
- An inventory is a list of **facts**. It does not say which fact is dangerous, or which one to fix first
- The ad-hoc alternative is to imagine attacks, which reliably finds the attacks you already know
- Today's question: how do you reason about a machine **completely**, rather than by inspiration

A method you can be systematic with beats a method you have to be clever with.

Model First, or Justify Later

- A model built **after** an exploit explains the exploit. It is a narrative, and it is always convincing
- A model built **first** forces questions about things you have not tried, which is where the unplanned attacks live
- The tooling is identical either way. The order is what changes the result
- It is why the model comes before every attack here, and why the capstone models an untouched machine

If the model contains only what you already did, you have written a report, not a model.

What a Threat Model Actually Is

- The **assets**, the **actors** who want them, the **entry points**, and a **priority order**
- An artefact, not a mood: something you hand to whoever operates the machine, and that they can argue with
- It is falsifiable, provided the attacker has a stated position. Claims about a vague adversary cannot be tested
- Ours, all semester: an unprivileged non-owner **foothold** as `svc-report`. **Not a general sudoer**, not in `docker`

The owner is out of scope by design: student is a full administrator, so modelling them attacking their own machine proves nothing.

Three Instruments, Three Jobs

- **Data-flow diagram.** Structure: what talks to what, and where the privilege changes
- **STRIDE.** Completeness: six questions asked of every element, including the ones you would skip
- **Attack tree.** Paths and cost: how single weaknesses compose, and which route is cheapest
- They overlap on purpose. Each one surfaces something the other two hide

Structure, completeness, cost. An instrument that is doing none of those three jobs is decoration.

Data-Flow Diagrams

Before you can ask what goes wrong, what exactly is talking to what?

Four Elements, One Diagram

- **External entity.** Something outside your control: a remote client, a person at a keyboard
- **Process.** Running code with an identity: a service, a scheduled job, a shell
- **Data store.** Data at rest: a configuration file, a script, a binary on disk
- **Data flow.** An arrow, labelled with what moves and when it moves
- Everything on the machine is one of the four. If it fits none of them, you have not yet decided what it is

The identity a process runs as is part of the element. svc-report and root are different processes even when the code is the same.

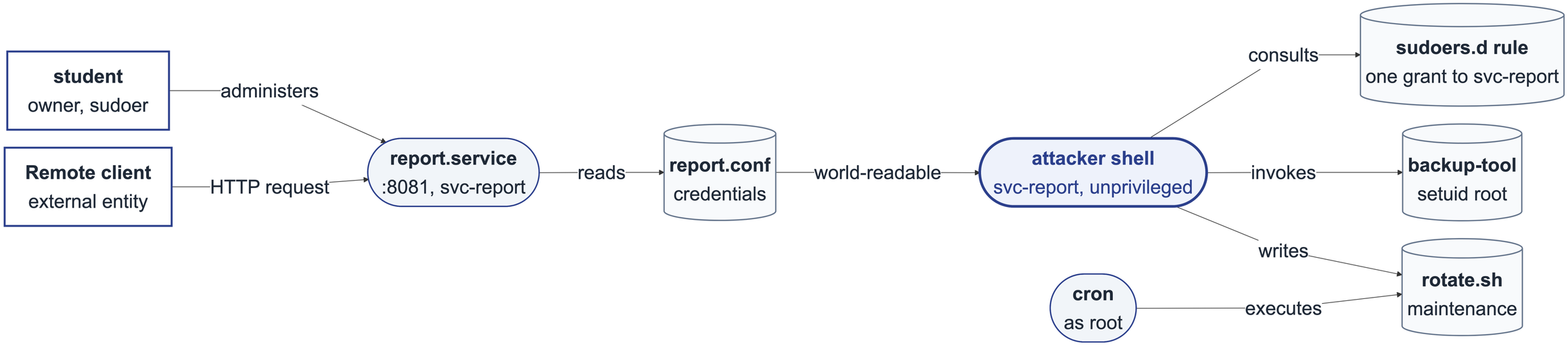
Trust Boundaries

- A dashed line drawn wherever data crosses a change of **privilege or trust**
- Not a network concept: `student` to `root` is a boundary on one machine with no network in sight
- Threats concentrate on crossings, because a crossing is where one side starts trusting the other's input
- A diagram without boundaries is an architecture drawing. The boundaries are what make it a security artefact

Miss a boundary and every threat that lived on it disappears from the model, silently.

The Machine, as a Diagram

- Every element came out of your own Lab 1 inventory: what listens, what is scheduled, what is privileged
- The service runs as `svc-report`, not `root`, and that one fact shapes half the model
- The attacker's shell is **inside** the picture: a foothold is an element, not an event

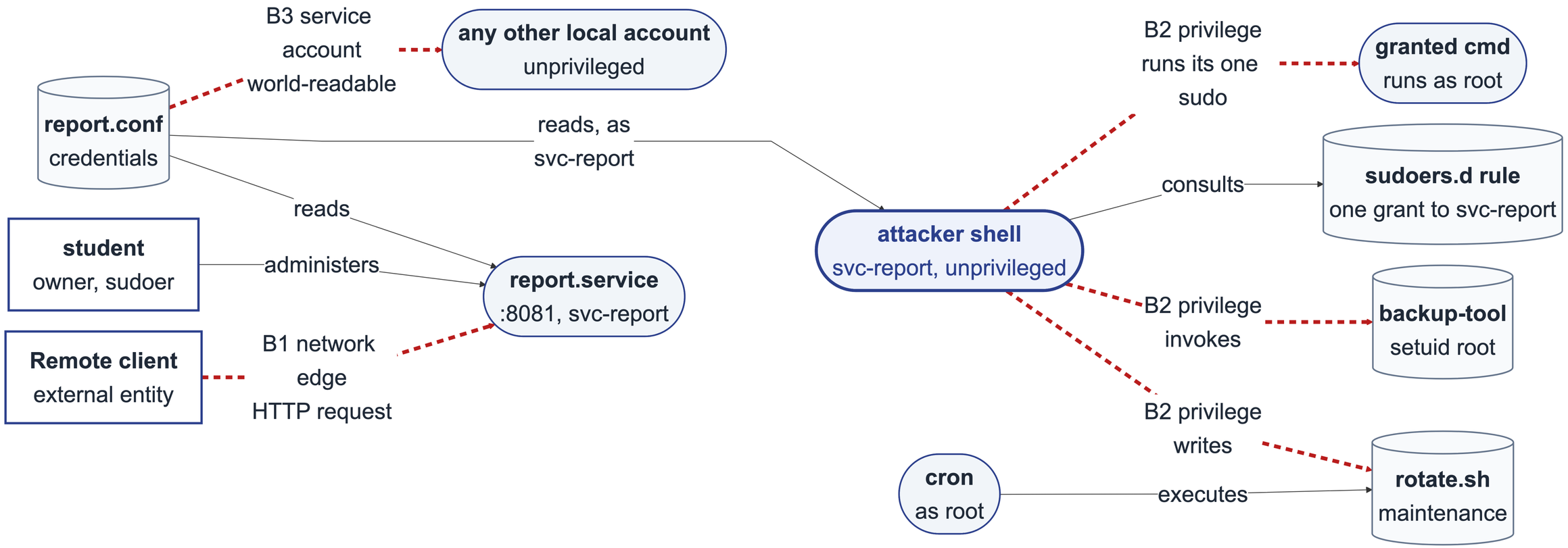


Draw the identity on every process. Who runs what is half the content of this diagram.

Where the Boundaries Fall

- **B1, the network edge.** One flow crosses it, which is why what listens is the first question
- **B2, privilege.** Three crossings, and all three start from the unprivileged shell
- **B3, the service account.** Any other local account crosses it with no exploit: the credential is world-readable

The Same Machine, Boundaries Marked



Count the crossings, not the boxes. That number is the size of the problem you actually have.

Lab Core

How a Diagram Goes Wrong

- **Too much detail.** Every file on the disk is not a data store. Draw what an attacker can reach or change
- **No boundaries.** The commonest failure, and the one that removes the entire point of the exercise
- **Store confused with process.** `backup-tool` on disk is a store; `backup-tool` running as root is a process
- **No scope statement.** Write down what is in and what is out, or the diagram cannot be finished

Scope is a claim. Writing down what you excluded is what lets somebody else check that you excluded it fairly.

Study Support

STRIDE

How do you find the threat you would never have thought to look for?

Six Questions, Asked of Everything

- **S**poofing: can something claim an identity that is not its own
- **T**ampering: can something change data or code that it should not
- **R**epudiation: can an action be denied afterwards, because nothing recorded it
- **I**nformation disclosure: can something read what it should not
- **D**enial of service: can something remove availability
- **E**levation of privilege: can something end up holding rights it was never given

Six is few enough to ask of every element, which is the only reason the method survives contact with a real system.

Every Category Is a Property Violated

- Spoofing violates **authenticity**; tampering violates **integrity**; information disclosure violates **confidentiality**
- Denial of service violates **availability**; elevation of privilege violates **authorisation**
- Repudiation is the odd one out: it violates **accountability**, the property Lecture 1 said CIA leaves out
- So the taxonomy is not arbitrary. It is last week's list of properties, turned round to face the attacker

If you can name the property, you can name the category. They are the same list read from opposite ends.

The Grid Beats the Flat List

Element	S	T	R	I	D	E
report.service :8081	.	.	X	X	X	.
Remote client	.	.	X	.	.	.
student (owner, n/a)	.	.	.	.	.	.
rotate.sh (at rest)	.	X	.	.	.	.
cron (as root)	.	.	.	.	.	X
backup-tool (setuid)	.	.	.	.	.	X
sudoers.d: the grant	.	.	.	.	.	X
granted cmd (as root)	.	.	.	.	.	X
report.conf (at rest)	.	.	.	X	.	.
attacker shell	.	.	X	.	.	X
cron -> rotate.sh (fl)	.	X	.	.	X	.
svc -> client (flow)	.	X	.	X	X	.

- One row per element, one column per question, and **every cell** is filled or marked `n/a` with a reason
- The empty cells are the value: a blank you have to justify is a question you did not quietly skip
- `n/a` is a legitimate answer given one word of why. Silence is not an answer
- Extend it until every element has a row: **a flow is an element too**, and so is the attacker's own shell

A flat list stops when you run out of ideas. A grid stops when it is full.

A Worked Row: the Status Service

```
report.service (:8081), six questions

S  no authentication of any kind
T  writes nothing, reads a root-owned file
R  the handler logs nothing: no evidence
I  the reply carries db_user
D  single-threaded: one request at a time
E  runs as svc-report, not as root
```

- Six answers, and only one was on anybody's list before the grid asked for it
- **T** is a good `n/a`: empty because the service writes nothing, not because you forgot it
- **I** is the finding: the machine hands part of a credential to anyone who asks for it
- **R** is the one everybody skips: no log, no evidence, and Week 8 makes that a decision
- **E** limits the blast radius to `svc-report`, not `root`; that account's own grants are a separate row

A row is finished when every letter has a sentence, not when you have found something interesting.

Lab Core

When a Threat Is Also a Step

- The root schedule runs `/opt/maint/rotate.sh`, and that script sits where an unprivileged process can write it
- As a threat it is **Tampering**: something changes code that it does not own
- As a step it is **Elevation of privilege**: the change is executed later, by root, on a schedule
- One weakness, two cells, and the second is visible only because the grid asked all six questions

Record the chain as a chain. A threat that enables another threat is worth more than two unrelated rows.

A Read Bit Is a Security Control

- `report.conf` is world-readable, so every unprivileged process on the machine already holds the credential
- Information disclosure with no exploit, no bug and no vulnerability identifier. The permission **is** the weakness
- It is the cheapest attack there is: one `cat`, no privilege, and a trace indistinguishable from any other read
- Most of what this course hardens is of exactly this kind: a default that nobody ever decided

Ask of every file: who can read this, and did anybody actually choose that?

What STRIDE Does Not Do

- It enumerates **categories**, not exploits. Naming a threat correctly is not knowing how to realise it
- It does not rank. Every cell arrives weighing the same, which is never true of a real machine
- It over-generates. Six questions across twenty elements is a hundred and twenty cells, most of them `n/a`
- It says nothing about **composition**: which two threats chain into one path to the goal

A completeness aid, not an oracle. Ranking and chaining are the next instrument's job.

Study Support

Attack Trees and Priorities

Of everything that could go wrong, what will an attacker actually do first?

Attack surface so far: enumerated last week in four dimensions, and still not a number. Today it becomes one

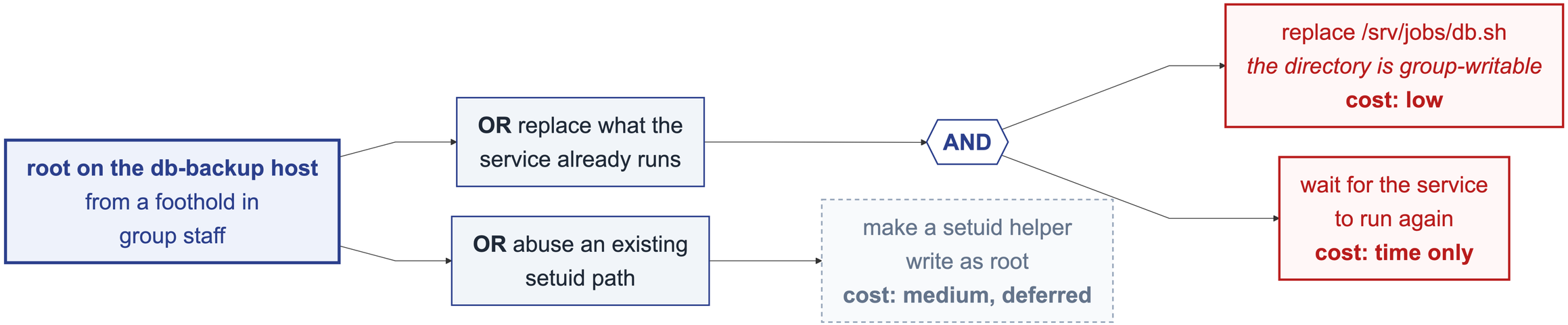
Attack Trees

- One goal at the root, decomposed downwards, with concrete actions at the leaves
- **OR** nodes are alternatives: any child achieves the parent. **AND** nodes are conjunctions: every child is needed
- Leaves carry a cost: effort, privilege required, time, noise, or a mechanism you do not have yet
- A tree is a claim about **paths.** The grid said what could go wrong; the tree says how the pieces fit together

Write leaves as actions somebody could perform on Tuesday, not as properties of the system.

A Tree for One Goal

- **Not your VM.** This is the `db-backup` host from Lecture 1, whose script is root-owned and whose directory is not
- Goal: `root` from a foothold in group `staff`. The upper branch is an **AND**
- Replacing the script achieves nothing until the service runs, and cost is written on the leaf
- A goal that does not name the attacker's **starting position** cannot be costed
- And a branch can be **closed** rather than deferred. Striking one out is analysis too



Same shape as your own machine, different mechanism. Building this tree for your VM is Part D of today's laboratory.

Lab Core

The Attacker Takes the Cheapest Path

- The cheapest complete path there: replace the script through the writable directory, wait, collect a root shell
- No exploit, no vulnerability identifier, no tool that is not already on the machine, and no privilege beyond the foothold
- Your machine has the **same shape** with a different mechanism: a writable file rather than a writable directory
- The interesting branch is rarely the cheapest one. Attackers are not curious, they are economical
- So the first defence you build removes the cheapest path, not the one that is most satisfying to build

Rank by the attacker's cost, not by your own interest. That is what makes an ordering defensible.

Leaves You Cannot Reach Yet

- The setuid branch is marked **deferred**: the mechanism arrives in Week 6, and today you can only name it
- A tree still works with deferred leaves, because its shape shows where risk sits before you can execute any of it
- This is the honest state of every real model. Nobody knows every mechanism on a machine they are asked to assess
- Mark them explicitly: a leaf marked deferred is analysis, while a leaf quietly left out is a hole in the argument

Deferred is a status, not a failure. The capstone in December asks you to come back and close them.

From Findings to a Ranked List

- Take the filled grid and the tree and produce one ordered list of weaknesses, most urgent first
- Rank as an argument: risk is roughly **likelihood times impact**, and both halves are estimates you are making up
- Resist the score. Two decimal places on a guess is false precision, and it hides the reasoning that matters
- What defends a rank is the sentence beside it, and your willingness to defend the first item against the second

A defensible order, argued, beats a correct order asserted. That is what report 1 is marked on.

Attack Surface as a Number

```
Metric: privileged paths and exposed
        secrets an unprivileged user can reach

  world-writable, run by root ..  1  rotate.sh
  setuid-root binaries .........  1  backup-tool
  credentials readable by all ..  1  report.conf
  passwordless sudo grants .....  1
  network services as root .....  0
                       total =    4
```

- Pick something **countable** and say what it counts, so two people get the same number
- The value on its own means little. The **delta** is the point: fix `rotate.sh` and the number becomes 3
- It should be monotone: a fix never raises it, a new exposed service never lowers it
- You report it in report 1, and again in December for a machine you have never seen

This is the running quantity of the whole unit. Every block from here shrinks it by one mechanism at a time.

From Model to Policy

- A threat model justifies a **policy**: the statement of what is allowed on this machine, and by whom
- Policy against mechanism was Lecture 1's seam. The model is how you get from threats to the statement
- One line, from today's tree: *no unprivileged user may modify anything that root executes*
- Everything after this week enforces statements of that shape: PAM, permissions, capabilities, MAC

Write the policy line before choosing the mechanism, or the mechanism quietly becomes the policy.

Today in the Laboratory

Lab 2 produces lab report 1, the first assessed deliverable. Bring your Lab 1 inventory: you cannot start without it.

- Attacker profile and ranked assets, then the diagram, the boundaries, and the grid extended to every element
- One attack tree for a goal you choose, with the cheapest path read off it
- A ranked remediation list, your defence of the top item, the surface metric, and the policy line your top fix assumes
- No root, no services, no exploits today: this is analysis, done in your group of three and written up individually

The model is the deliverable. It is marked on the argument, not on how many threats you can list.

Lab Core

Reading and Reference

- **A. Shostack, Threat Modeling: Designing for Security, Wiley, 2014**. The standard treatment of data-flow diagrams and STRIDE. Chapters 1 to 3 are today's material.
- **Microsoft, The STRIDE Threat Model**. The categories as originally stated, each paired with the property it violates. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
- **B. Schneier, Attack Trees, Dr. Dobb's Journal, 1999**. Where attack trees come from, with the cost annotations already in place. https://www.schneier.com/academic/archives/1999/12/attack_trees.html
- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. Chapter 14 for risk assessment, if the ranking argument is what interests you.
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. Chapter 1, for the same machine-level view the laboratory takes.

Reference

Key Takeaways

- Model first. A model built after the attack justifies the attack and discovers nothing
- Name the attacker and draw the boundaries. Everything else in the model hangs from those two decisions
- STRIDE gives completeness, the tree gives paths and cost, the ranking gives the argument
- The cheapest path is the one that gets used, so defend against economics rather than against ingenuity
- Your surface is four today. The rest of the unit is spent making that number fall

Report 1 is the model, not the findings. You are marked on the argument you are able to defend.